extraction_data

March 21, 2026

1 Extração

[1]:
```
import os, re, glob, csv
import pandas as pd
print('OK')
```

```
OK
```

[2]:
```
CORRECT_DIR   = './dataset/correct'     # arquivos _angles.txt
INCORRECT_DIR = './dataset/incorrect'   # arquivos _angles_inc.txt
OUTPUT_CSV    = './output/dataset.csv'
os.makedirs(os.path.dirname(OUTPUT_CSV), exist_ok=True)
```

[3]:
```
# Ordem exata da coluna 'Vicon Angles' da Tabela 2 (artigo PMC5773117)
VICON_ANGLE_JOINTS = [
    'Head',            # 1  - absoluto
    'Left_head',       # 2
    'Right_head',      # 3
    'Left_neck',       # 4
    'Right_neck',      # 5
    'Left_clavicle',   # 6
    'Right_clavicle',  # 7
    'Thorax',          # 8  - absoluto
    'Left_thorax',     # 9
    'Right_thorax',    # 10
    'Pelvis',          # 11 - absoluto
    'Left_pelvis',     # 12
    'Right_pelvis',    # 13
    'Left_hip',        # 14
    'Right_hip',       # 15
    'Left_femur',      # 16
    'Right_femur',     # 17
    'Left_knee',       # 18
    'Right_knee',      # 19
    'Left_tibia',      # 20
    'Right_tibia',     # 21
    'Left_ankle',      # 22
    'Right_ankle',     # 23
```

```
    'Left_foot',        # 24
    'Right_foot',       # 25
    'Left_toe',         # 26
    'Right_toe',        # 27
    'Left_shoulder',    # 28
    'Right_shoulder',   # 29
    'Left_elbow',       # 30
    'Right_elbow',      # 31
    'Left_radius',      # 32
    'Right_radius',     # 33
    'Left_wrist',       # 34
    'Right_wrist',      # 35
    'Left_upperhand',   # 36
    'Right_upperhand',  # 37
    'Left_hand',        # 38
    'Right_hand',       # 39
]

# Expande para 117 colunas: cada junta tem eixo X (Flex/Ext), Y (Abd/Add), Z␣
↪(Rot)
ANGLE_COLS = [f'{j}_{ax}' for j in VICON_ANGLE_JOINTS for ax in ['X', 'Y', 'Z']]

assert len(ANGLE_COLS) == 117
HEADER = ['movement', 'subject', 'execution', 'label', 'frame'] + ANGLE_COLS
print(f'{len(ANGLE_COLS)} colunas de ângulos | {len(HEADER)} colunas totais no␣
↪CSV')
```

```
117 colunas de ângulos | 122 colunas totais no CSV
```

[4]:
```
def parse_filename(filepath):
    m = re.match(r'm(\d+)_s(\d+)_e(\d+)_angles(_inc)?\.txt',
                 os.path.basename(filepath))
    if not m:
        raise ValueError(f'Nome fora do padrão: {filepath}')
    return int(m.group(1)), int(m.group(2)), int(m.group(3)), (1 if m.group(4)␣
↪else 0)


all_files = (
    sorted(glob.glob(os.path.join(CORRECT_DIR,   '*_angles.txt'))) +
    sorted(glob.glob(os.path.join(INCORRECT_DIR, '*_angles_inc.txt')))
)
print(f'Arquivos encontrados: {len(all_files)}')


with open(OUTPUT_CSV, 'w', newline='') as f:
    writer = csv.writer(f)
```

```
    writer.writerow(HEADER)

    for i, fp in enumerate(all_files):
        try:
            mov, subj, exe, label = parse_filename(fp)
            # processa um arquivo por vez - não acumula na memória
            for frame_idx, row in enumerate(pd.read_csv(fp, header=None).
↪values):
                writer.writerow([mov, subj, exe, label, frame_idx] + row.
↪tolist())
        except Exception as e:
            print(f'Erro em {fp}: {e}')

        if (i + 1) % 50 == 0:
            print(f'  {i+1}/{len(all_files)} arquivos processados...')

print(f'\nSalvo em: {OUTPUT_CSV}')
```

```
Arquivos encontrados: 1980
  50/1980 arquivos processados…
  100/1980 arquivos processados…
  150/1980 arquivos processados…
  200/1980 arquivos processados…
  250/1980 arquivos processados…
  300/1980 arquivos processados…
  350/1980 arquivos processados…
  400/1980 arquivos processados…
  450/1980 arquivos processados…
  500/1980 arquivos processados…
  550/1980 arquivos processados…
  600/1980 arquivos processados…
  650/1980 arquivos processados…
  700/1980 arquivos processados…
  750/1980 arquivos processados…
  800/1980 arquivos processados…
  850/1980 arquivos processados…
  900/1980 arquivos processados…
  950/1980 arquivos processados…
  1000/1980 arquivos processados…
  1050/1980 arquivos processados…
  1100/1980 arquivos processados…
  1150/1980 arquivos processados…
  1200/1980 arquivos processados…
  1250/1980 arquivos processados…
  1300/1980 arquivos processados…
  1350/1980 arquivos processados…
  1400/1980 arquivos processados…
```

```
  1450/1980 arquivos processados…
  1500/1980 arquivos processados…
  1550/1980 arquivos processados…
  1600/1980 arquivos processados…
  1650/1980 arquivos processados…
  1700/1980 arquivos processados…
  1750/1980 arquivos processados…
  1800/1980 arquivos processados…
  1850/1980 arquivos processados…
  1900/1980 arquivos processados…
  1950/1980 arquivos processados…

Salvo em: ./output/dataset.csv
```

[5]:
```python
df = pd.read_csv(OUTPUT_CSV)

print(f'Shape do dataset: {df.shape}')
```

```
Shape do dataset: (485446, 122)
```

[6]:
```python
print(df.columns.tolist())
```

```
['movement', 'subject', 'execution', 'label', 'frame', 'Head_X', 'Head_Y',
'Head_Z', 'Left_head_X', 'Left_head_Y', 'Left_head_Z', 'Right_head_X',
'Right_head_Y', 'Right_head_Z', 'Left_neck_X', 'Left_neck_Y', 'Left_neck_Z',
'Right_neck_X', 'Right_neck_Y', 'Right_neck_Z', 'Left_clavicle_X',
'Left_clavicle_Y', 'Left_clavicle_Z', 'Right_clavicle_X', 'Right_clavicle_Y',
'Right_clavicle_Z', 'Thorax_X', 'Thorax_Y', 'Thorax_Z', 'Left_thorax_X',
'Left_thorax_Y', 'Left_thorax_Z', 'Right_thorax_X', 'Right_thorax_Y',
'Right_thorax_Z', 'Pelvis_X', 'Pelvis_Y', 'Pelvis_Z', 'Left_pelvis_X',
'Left_pelvis_Y', 'Left_pelvis_Z', 'Right_pelvis_X', 'Right_pelvis_Y',
'Right_pelvis_Z', 'Left_hip_X', 'Left_hip_Y', 'Left_hip_Z', 'Right_hip_X',
'Right_hip_Y', 'Right_hip_Z', 'Left_femur_X', 'Left_femur_Y', 'Left_femur_Z',
'Right_femur_X', 'Right_femur_Y', 'Right_femur_Z', 'Left_knee_X', 'Left_knee_Y',
'Left_knee_Z', 'Right_knee_X', 'Right_knee_Y', 'Right_knee_Z', 'Left_tibia_X',
'Left_tibia_Y', 'Left_tibia_Z', 'Right_tibia_X', 'Right_tibia_Y',
'Right_tibia_Z', 'Left_ankle_X', 'Left_ankle_Y', 'Left_ankle_Z',
'Right_ankle_X', 'Right_ankle_Y', 'Right_ankle_Z', 'Left_foot_X', 'Left_foot_Y',
'Left_foot_Z', 'Right_foot_X', 'Right_foot_Y', 'Right_foot_Z', 'Left_toe_X',
'Left_toe_Y', 'Left_toe_Z', 'Right_toe_X', 'Right_toe_Y', 'Right_toe_Z',
'Left_shoulder_X', 'Left_shoulder_Y', 'Left_shoulder_Z', 'Right_shoulder_X',
'Right_shoulder_Y', 'Right_shoulder_Z', 'Left_elbow_X', 'Left_elbow_Y',
'Left_elbow_Z', 'Right_elbow_X', 'Right_elbow_Y', 'Right_elbow_Z',
'Left_radius_X', 'Left_radius_Y', 'Left_radius_Z', 'Right_radius_X',
'Right_radius_Y', 'Right_radius_Z', 'Left_wrist_X', 'Left_wrist_Y',
'Left_wrist_Z', 'Right_wrist_X', 'Right_wrist_Y', 'Right_wrist_Z',
'Left_upperhand_X', 'Left_upperhand_Y', 'Left_upperhand_Z', 'Right_upperhand_X',
'Right_upperhand_Y', 'Right_upperhand_Z', 'Left_hand_X', 'Left_hand_Y',
'Left_hand_Z', 'Right_hand_X', 'Right_hand_Y', 'Right_hand_Z']
```

[7]:
```python
print(df.isnull().sum())
print(f'\nTotal de valores nulos: {df.isnull().sum().sum()}')

display(df.describe().iloc[:, :10])
```

```
movement        0
subject         0
execution       0
label           0
frame           0
               ..
Left_hand_Y     0
Left_hand_Z     0
Right_hand_X    0
Right_hand_Y    0
Right_hand_Z    0
Length: 122, dtype: int64

Total de valores nulos: 0
```

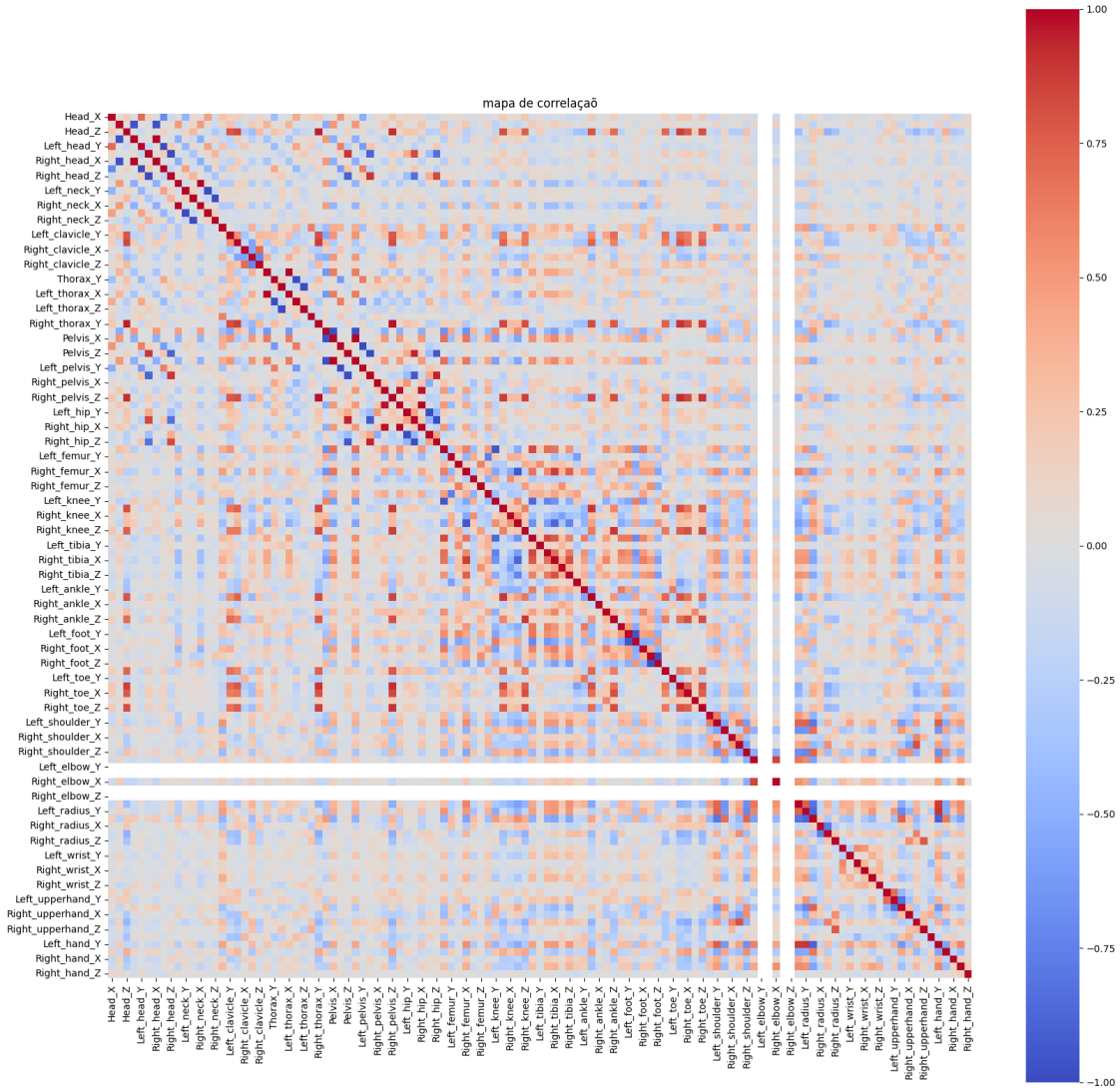
	movement	subject	execution	label
count	485446.000000	485446.000000	485446.000000	485446.000000
mean	5.310943	5.394886	5.570830	0.500387
std	2.828780	2.813642	2.837304	0.500000
min	1.000000	1.000000	1.000000	0.000000
25%	3.000000	3.000000	3.000000	0.000000
50%	5.000000	5.000000	6.000000	1.000000
75%	8.000000	8.000000	8.000000	1.000000
max	10.000000	10.000000	10.000000	1.000000

	frame	Head_X	Head_Y	Head_Z
count	485446.000000	485446.000000	485446.000000	485446.000000
mean	130.558313	0.667901	-8.259117	-0.901035
std	85.812854	7.970750	14.842713	27.555822
min	0.000000	-87.051100	-51.755800	-82.057400
25%	61.000000	-2.017290	-17.129475	-5.842887
50%	122.000000	0.708425	-11.161600	0.104114
75%	187.000000	3.145810	-3.432825	4.504450
max	581.000000	69.702100	100.839000	102.857000

	Left_head_X	Left_head_Y
count	485446.000000	485446.000000
mean	8.102230	0.460678
std	14.655656	8.312591
min	-100.728000	-80.612100
25%	3.219817	-2.233478
50%	10.472050	0.694130
75%	16.877900	3.078588

```
max        78.748300       68.916000
```

[8]:
```python
import seaborn as sns

import matplotlib.pyplot as plt


corr = df[ANGLE_COLS].corr()

# Plot heatmap
plt.figure(figsize=(20, 20))
sns.heatmap(corr, annot=False, cmap='coolwarm', square=True)
plt.title('mapa de correlaçaõ')
plt.show()
```



[9]:
```
angle_variation = (
    df.groupby(['movement', 'subject', 'execution'])[ANGLE_COLS]
    .agg(lambda x: x.max() - x.min())   # amplitude de cada execução individual
    .groupby('movement')                  # agrupa por movimento
    .mean()                               # média das amplitudes
)

max_variation_per_exe = angle_variation.idxmax(axis=1)
max_value_per_exe     = angle_variation.max(axis=1)

result = pd.DataFrame({
    'max_angle'    : max_variation_per_exe,
    'max_variation': max_value_per_exe.round(2)
})
display(result)
```

```
               max_angle  max_variation
movement
1          Right_tibia_X         134.47
2          Right_tibia_X          91.44
3          Right_tibia_X         102.37
4          Right_femur_X          81.01
5          Right_tibia_X         111.47
6            Right_toe_Y          81.84
7         Right_radius_Z         154.78
8           Right_hand_X          71.32
9         Right_radius_Z         170.76
10          Right_hand_Z         145.92
```

[10]:
```
import numpy as np
import os

df_corretos = df[df['label'] == 0].copy()
df_incorretos = df[df['label'] == 1].copy()

def gerar_mock_data(df_base, quantidade_desejada):
    execucoes = [group for _, group in df_base.groupby(['movement', 'subject',␣
 ↪'execution'])]
    novas_execucoes = []
    for i in range(quantidade_desejada):
        base = execucoes[np.random.randint(0, len(execucoes))].copy()
        ruido = np.random.normal(0, 0.01, size=(len(base), len(ANGLE_COLS)))
        base[ANGLE_COLS] = base[ANGLE_COLS] + ruido
        base['subject'] = base['subject'] + 2000 + i
        base['execution'] = base['execution'] + 2000 + i
        novas_execucoes.append(base)
    return pd.concat(novas_execucoes)
```

```python
print("Gerando novos dados..")
mock_corretos = gerar_mock_data(df_corretos, 100)
mock_incorretos = gerar_mock_data(df_incorretos, 100)

# Junta o original com o mock
df_final = pd.concat([df, mock_corretos, mock_incorretos]).
 ↪reset_index(drop=True)
df = df_final

CAMINHO_FINAL = './output/dataset_final_5000.csv'
df.to_csv(CAMINHO_FINAL, index=False)

print(f"success")
print(f"O arquivo foi salvo em: {CAMINHO_FINAL}")
print(f"Tamanho final do arquivo: {df.shape[0]} linhas (frames).")
```

```
Gerando novos dados..
success
O arquivo foi salvo em: ./output/dataset_final_5000.csv
Tamanho final do arquivo: 535117 linhas (frames).
```

[12]:
```python
print(df['subject'].unique())
```

```
[   1    2    3    4    5    6    7    8    9   10 2002 2003 2011 2007
 2009 2016 2015 2012 2017 2014 2022 2018 2027 2026 2023 2030 2025 2034
 2035 2029 2032 2037 2038 2043 2040 2046 2048 2044 2051 2053 2049 2056
 2052 2054 2061 2065 2063 2059 2068 2072 2066 2071 2073 2074 2079 2076
 2078 2075 2085 2083 2081 2086 2082 2094 2091 2093 2089 2098 2097 2101
 2102 2106 2107 2001 2005 2006 2008 2010 2028 2033 2036 2041 2042 2057
 2062 2060 2069 2067 2080 2084 2088 2087 2095 2104 2105]
```

[15]:
```python
# Análise dos metadados do novo dataset
print("=== ANÁLISE DO DATASET FINAL ===\n")

# Contagem de registros por label
print("Distribuição por label (0=correto, 1=incorreto):")
print(df['label'].value_counts().sort_index())
print(f"Total: {len(df)} frames\n")

# Movimentos únicos
print(f"Movimentos únicos: {sorted(df['movement'].unique())}")
print(f"Total de movimentos: {df['movement'].nunique()}\n")

# Sujeitos únicos
print(f"Sujeitos únicos: {df['subject'].nunique()}")
print(f"Range: {df['subject'].min()} - {df['subject'].max()}\n")
```

```python
# Execuções por movimento
print("Média de execuções por movimento:")
exec_por_mov = df.groupby('movement')['execution'].nunique()
print(exec_por_mov)
print(f"Média: {exec_por_mov.mean():.2f}\n")

# Breakdown completo
print("Breakdown por movement e label:")
breakdown = df.groupby(['movement', 'label']).size().unstack(fill_value=0)
breakdown['total'] = breakdown.sum(axis=1)
print(breakdown)
```

```
=== ANÁLISE DO DATASET FINAL ===

Distribuição por label (0=correto, 1=incorreto):
label
0    267531
1    267586
Name: count, dtype: int64
Total: 535117 frames

Movimentos únicos: [np.int64(1), np.int64(2), np.int64(3), np.int64(4),
np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(10)]
Total de movimentos: 10

Sujeitos únicos: 95
Range: 1 - 2107

Média de execuções por movimento:
movement
1     27
2     26
3     30
4     34
5     25
6     28
7     31
8     28
9     28
10    30
Name: execution, dtype: int64
Média: 28.70

Breakdown por movement e label:
label         0      1  total
movement
1         29828  30667  60495
2         24847  24650  49497
```

```
3          26824  25983  52807
4          31893  29815  61708
5          30744  37170  67914
6          22599  21923  44522
7          26962  26965  53927
8          24456  22173  46629
9          25693  26804  52497
10         23685  21436  45121
```

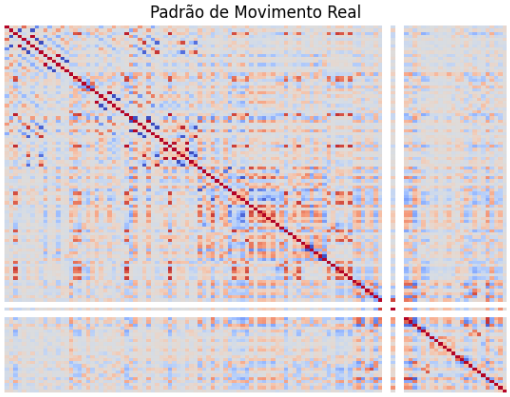
[17]:
```python
# Separar dados reais e mock
real_df = df[df['subject'] < 1000]
mock_df = df[df['subject'] >= 1000]

print("=== COMPARAÇÃO DE MOVIMENTOS: REAL vs MOCK ===\n")

# Breakdown para dados reais
print("Breakdown REAL (subject < 1000):")
breakdown_real = real_df.groupby(['movement', 'label']).size().
 ↪unstack(fill_value=0)
breakdown_real['total'] = breakdown_real.sum(axis=1)
print(breakdown_real)
print(f"Total frames reais: {len(real_df)}\n")

# Breakdown para dados mock
print("Breakdown MOCK (subject >= 1000):")
breakdown_mock = mock_df.groupby(['movement', 'label']).size().
 ↪unstack(fill_value=0)
breakdown_mock['total'] = breakdown_mock.sum(axis=1)
print(breakdown_mock)
print(f"Total frames mock: {len(mock_df)}\n")

# Comparação de execuções por movimento
exec_real = real_df.groupby('movement')['execution'].nunique()
exec_mock = mock_df.groupby('movement')['execution'].nunique()

print("Execuções por movimento:")
print("REAL:")
print(exec_real)
print("\nMOCK:")
print(exec_mock)
print("\nDiferença (Mock - Real):")
print(exec_mock - exec_real)
```

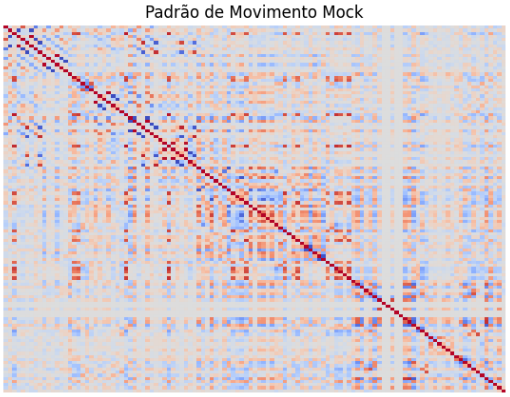
```
=== COMPARAÇÃO DE MOVIMENTOS: REAL vs MOCK ===

Breakdown REAL (subject < 1000):
label         0      1  total
movement
```

```
1          27063  28116  55179
2          23073  22819  45892
3          23717  23255  46972
4          27884  27014  54898
5          28928  33749  62677
6          21146  19842  40988
7          24917  23275  48192
8          21802  20558  42360
9          23286  24426  47712
10         20719  19857  40576
Total frames reais: 485446

Breakdown MOCK (subject >= 1000):
label        0     1  total
movement
1          2765  2551   5316
2          1774  1831   3605
3          3107  2728   5835
4          4009  2801   6810
5          1816  3421   5237
6          1453  2081   3534
7          2045  3690   5735
8          2654  1615   4269
9          2407  2378   4785
10         2966  1579   4545
Total frames mock: 49671

Execuções por movimento:
REAL:
movement
1     10
2     10
3     10
4     10
5     10
6     10
7     10
8     10
9     10
10    10
Name: execution, dtype: int64

MOCK:
movement
1     17
2     16
3     20
4     24
```

```
5     15
6     18
7     21
8     18
9     18
10    20
Name: execution, dtype: int64

Diferença (Mock - Real):
movement
1      7
2      6
3     10
4     14
5      5
6      8
7     11
8      8
9      8
10    10
Name: execution, dtype: int64
```

[29]:
```
corr_real = df[df['subject'] < 1000][ANGLE_COLS].corr()
corr_mock = df[df['subject'] >= 1000][ANGLE_COLS].corr()

plt.figure(figsize=(15, 5))
plt.subplot(1, 2, 1)
sns.heatmap(corr_real, cbar=False, cmap='coolwarm', xticklabels=False,␣
 ↪yticklabels=False)
plt.title("Padrão de Movimento Real")

plt.subplot(1, 2, 2)
sns.heatmap(corr_mock, cbar=False, cmap='coolwarm', xticklabels=False,␣
 ↪yticklabels=False)
plt.title("Padrão de Movimento Mock")
plt.show()
```


Padrão de Movimento Real


Padrão de Movimento Mock